



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-27145 Go crypto/x509 hostname-verification CPU exhaustion risk

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-07
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-27145 Go crypto/x509 hostname-verification CPU exhaustion risk - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-07 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
10. Threat Hunting
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. References

CVE-2026-27145 Go crypto/x509 hostname-verification CPU exhaustion risk

1. Executive Summary

CVE-2026-27145 is a medium-severity algorithmic-complexity flaw in Go's standard-library `crypto/x509` package. The issue affects hostname verification logic: `(\*x509.Certificate).VerifyHostname` repeatedly split the same candidate hostname while iterating over DNS Subject Alternative Name (SAN) entries, causing verification cost to grow quadratically as SAN count and hostname label count increased.

The practical risk is CPU exhaustion or denial of service rather than memory corruption or code execution. Public Go, CVE, and NVD materials indicate the cost is incurred even for untrusted certificates because `x509.Verify` performs hostname validation before certificate-chain building. Exposure is therefore most relevant to Go TLS clients, proxies, agents, and applications that verify attacker-controlled certificates or directly call `x509.Verify` / `VerifyHostname` with untrusted `DNSName` input.

2. Vulnerability Metadata

Field	Value
CVE	`CVE-2026-27145`
Go advisory ID	`GO-2026-5037`
Product / component	Go standard library `crypto/x509`
Affected routines called out in the CVE record	`HostnameError.Error`, `matchHostnames`, `Certificate.Verify`, and `Certificate.VerifyHostname`
Weakness classification	`CWE-407: Inefficient Algorithmic Complexity`
CVSS v3.1	`6.5 / MEDIUM`
CVSS vector	`CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:N/I:L/A:H`
Affected version branches in the published CVE record	versions before `Go 1.25.11`, and `Go 1.26.0` through before `Go 1.26.4`
Fixed versions	`Go 1.25.11` and `Go 1.26.4`
Public disclosure date in Go / NVD sources	`2026-06-02`
Upstream tracking	Go issue `#79694` and Gerrit change `783621`
Credit	Jakub Ciolek

3. Source Review & Confidence

Source	Contribution	Confidence
Reviewed source	MSRC lists `CVE-2026-27145` and served as the originating intake item for this review.	Useful as the intake and publication reference, but not the richest technical source.
Reviewed source	The Go CNA CVE 5.2 record describes repeated hostname splitting in `crypto/x509`, assigns `CWE-407`, lists	Primary authoritative vulnerability record for this issue.

	affected vers	
Reviewed source	NVD records `CVE-2026-27145` as published on 2026-06-02 and exposes CVSS v3.1 `6.5 / MEDIUM` with vector `AV:N/AC:H/PR:N`	Strong corroborating source for severity and publication metadata.
Reviewed source	The upstream Go issue explains that `VerifyHostname` repeatedly split the same hostname, that cost scales with SAN count	Strong source for exploitation framing and operational exposure context.
Reviewed source	The upstream fix commit states that `crypto/x509` now splits the candidate hostname only once, links the change to issue	Primary remediation source.
Reviewed source	Go release history states that `go1.25.11` and `go1.26.4`, both released 2026-06-02, include security fixes to `crypto/x`	Primary release-validation source.
Reviewed source	Debian tracks the same `crypto/x509` hostname-verification inefficiency and links to upstream and ecosystem references	Good downstream corroboration source.
Reviewed source	Ubuntu publicly tracks the CVE with medium priority and evaluation status across multiple Go package lines, confirming d	Useful corroborating source for downstream packaging and prioritization context.

Confidence is high on the technical core of the issue because multiple strong sources align: Confidence is lower on exploit prevalence because no reviewed source showed CISA KEV inclusion, confirmed in-the-wild exploitation, or a public weaponized proof of concept. The strongest supportable conclusion is that this is a real but situational CPU-exhaustion risk whose impact depends on certificate parsing paths, attacker control over certificates or hostnames, and request volume.

4. Vulnerability Details

According to the Go CNA and the upstream fix record, `VerifyHostname` previously called `matchHostnames` in a loop over all DNS SAN entries. Each loop iteration repeated `strings.Split(host, ".")` on the same candidate hostname. That duplicated work made verification cost scale with both:

- the number of DNS SAN entries carried by the certificate, and
- the size or label count of the hostname being verified.

The upstream issue explains why this mattered operationally: `x509.Verify` performs hostname validation before chain building, so the expensive work could be forced even when the presented certificate was ultimately untrusted. In practice, an attacker who can induce a target to process certificates with very large DNS SAN lists can amplify CPU consumption during verification.

The issue is primarily a performance and availability problem. Reviewed public material does not support claims of remote code execution, authentication bypass, or secret disclosure from this flaw alone.

5. Attack Scenarios

1. A malicious or compromised TLS server presents a certificate containing a very large DNS SAN list to a Go client. Repeated verification attempts force high CPU cost during hostname checking, degrading the client or intermediary that performs certificate validation.

2. A Go-based proxy, scanner, agent, or middleware component repeatedly verifies attacker-controlled certificates at scale. Even invalid certificates still incur the hostname-check overhead before chain rejection.
3. An application directly calls `VerifyHostname` or `x509.Verify` with attacker-controlled `DNSName` values and does not enforce normal DNS length limits, allowing unusually long multi-label hostnames to increase per-verification work further.
4. Multi-tenant or internet-facing services that initiate large numbers of outbound TLS validations can be pressured into sustained CPU burn if an attacker controls endpoints or certificates involved in those validations.

6. Threat Landscape & Exploitation Status

- Public disclosure is broad across MSRC, the Go CNA, NVD, GitHub advisory aggregators, Debian, Ubuntu, and CISA's weekly bulletin.
- The CVE record includes CISA ADP SSVC content showing `Exploitation: none`, `Automatable: no`, and `Technical Impact: partial` at the time of enrichment.
- No evidence was collected during this review showing CISA KEV inclusion or confirmed active exploitation in the wild.
- Attack complexity is not trivial: the NVD/CISA ADP CVSS vector scores the issue as `AC:H`, which is consistent with a specialized denial-of-service path rather than a low-effort internet wormable flaw.
- The risk is still material for certificate-heavy workflows, especially clients or services that validate many attacker-influenced certificates or accept uncontrolled hostname input.

7. Affected Products and Exposure

The most defensible exposure statement is that affected products are Go applications built with vulnerable releases of the Go standard library and using `crypto/x509` hostname-verification paths.

Higher-risk patterns:

- Go TLS clients that connect to untrusted or attacker-reachable remote endpoints.
- Security tools, crawlers, proxies, mail gateways, agents, and middleware that perform large volumes of certificate validation.
- Applications that directly invoke `x509.Verify` with `VerifyOptions.DNSName` or call `VerifyHostname` on attacker-controlled inputs.
- Environments where remote parties can influence certificate contents or induce repeated TLS handshakes.

Lower-risk patterns:

- Applications rebuilt on `Go 1.25.11`, `Go 1.26.4`, or later maintained releases.
- Systems that do not process attacker-controlled certificates or that rarely perform hostname verification.
- Deployments that already constrain outbound TLS destinations, rate-limit expensive verification paths, or normalize hostname input before verification.

Downstream exposure should also include distro-packaged Go runtimes and embedded products that lag upstream updates; Ubuntu and Debian are both tracking the CVE in their public vulnerability portals.

8. Mitigation and Workarounds

Priority actions:

5. Upgrade Go toolchains and rebuild affected applications to `1.25.11`, `1.26.4`, or later maintained releases.
6. Inventory Go-built clients, proxies, agents, and certificate-processing services that verify remote certificates at scale.
7. Where applications pass externally derived `DNSName` values into verification routines, enforce sane hostname length and formatting limits before verification.
8. Add rate limiting, concurrency limits, or circuit breakers around certificate-verification-heavy code paths if immediate upgrades are not possible.
9. Review whether multi-tenant or internet-facing systems can be induced to perform repeated outbound verifications against attacker-controlled endpoints.

10. Prefer upstream security releases rather than trying to locally patch around the algorithmic behavior.

9. Detection Engineering

Detection is primarily about vulnerable-version discovery and performance-path visibility:

Signal	Telemetry Source	Analytic Goal
Inventory Go versions used in outbound TLS clients, scanners, agents, proxies, and service-to-service middleware.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Flag systems running versions earlier than `1.25.11` or in the vulnerable `1.26.0`-`1.26.3` range.	Asset inventory, SBOM, package management, CI/CD telemetry	Find vulnerable builds or packages
Review CPU and latency anomalies around certificate-validation or TLS-handshake code paths, especially when interacting with untrusted endpoints.	Application logs, service metrics, EDR / crash telemetry	Spot exploitation symptoms or instability
Instrument certificate-processing services to log certificate SAN counts, handshake timing, and retry spikes where operationally safe.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity
Hunt for workloads that make unusually high numbers of outbound TLS connections to attacker-influenced infrastructure.	Asset inventory / service / security telemetry	Identify vulnerable systems, exposure, or suspicious follow-on activity

9.1 Detection Query

```
// Build and package provenance pivots for CVE-2026-27145 Go crypto/x509 hostname-verification CPU
// exhaustion risk
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ProcessCommandLine has_any ("go version", "go env", "pip show", "pip list", "syft", "grype",
"sbom")
| project Timestamp, DeviceName, InitiatingProcessAccountName, FileName, ProcessCommandLine
| order by Timestamp desc
```

9.2 Exposure / Triage Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

10. Threat Hunting

Suggested hunt questions:

- Which Go applications in the environment were built with versions older than `1.25.11` or in the vulnerable `1.26.0`-`1.26.3` window?
- Which of those applications validate certificates from untrusted or semi-trusted remote parties?
- Are there recent CPU spikes, timeout bursts, or handshake-latency increases tied to TLS verification activity?

- Do any systems ingest or generate certificates with unusually large DNS SAN lists?
- Are internal scanners, collectors, or proxy layers repeatedly contacting the same suspicious endpoints while showing certificate-validation slowdown?
- Do any applications pass user-supplied hostname strings into `x509.Verify` or `VerifyHostname` without length controls?

10.1 Hunt Query

```
// Hunt for service instability or repeated process failures near affected build lineage
DeviceProcessEvents
| where Timestamp > ago(30d)
| where ActionType has_any ("ProcessTerminated", "Crash", "ApplicationError") or ProcessCommandLine
has_any ("panic", "fatal error")
| project Timestamp, DeviceName, FileName, ProcessCommandLine, InitiatingProcessFileName
| order by Timestamp desc
```

11. MITRE ATT&CK Mapping

No ATT&CK technique is assigned with high confidence for this intake review. The public evidence supports an algorithmic-complexity / denial-of-service condition in certificate verification, but not a documented adversary procedure chain specific enough to justify a precise ATT&CK mapping without over-claiming.

12. Validation / Lab Testing

This intake review validated the report against published advisory, API, and upstream development material rather than reproducing the issue locally. Analyst validation steps completed:

- Reviewed the MSRC Update Guide listing that triggered the intake.
- Queried the published CVE 5.2 record from the Go CNA for the authoritative description, version ranges, routine names, references, and CWE assignment.
- Queried the NVD JSON 2.0 API for severity, vector, and publication metadata.
- Reviewed the upstream Go issue `#79694` for exposure details and impact framing.
- Reviewed the upstream fix commit for remediation wording and disclosure linkage.
- Reviewed public release-history text confirming fixes in `Go 1.25.11` and `Go 1.26.4`.
- Reviewed Debian and Ubuntu public tracker material for downstream packaging awareness.

No Lost Boys Cyber proof-of-concept execution was performed as part of this draft review.

13. Recommended Actions Summary

Priority	Owner	Action
Critical	Platform / Application Owners	Upgrade affected Go toolchains to `1.25.11`, `1.26.4`, or later and rebuild exposed applications.
High	Security / Platform Owners	Prioritize Go clients, proxies, agents, and middleware that verify attacker-reachable certificates.
Critical	Platform / Application Owners	Add hostname-input limits and operational safeguards around certificate-verification-heavy code paths where patching is delayed.
High	SOC / Detection Engineering	Hunt for certificate-validation-related CPU spikes and suspiciously large SAN-bearing certificates in relevant telemetry.
High	Third-Party Risk / Asset Management	Track downstream distro packaging status

		if production dependencies come from Ubuntu, Debian, or vendor images rather than direct upstream Go installs.
--	--	--

14. References

11. Microsoft Security Response Center, `CVE-2026-27145`
12. CVE 5.2 record for `CVE-2026-27145` (Go CNA)
13. NVD entry / JSON 2.0 API for `CVE-2026-27145`
14. Go issue tracker `#79694`
15. Go commit fixing `crypto/x509` hostname parsing behavior
16. Go release history for `1.25.11`
17. Go release history for `1.26.4`
18. Debian Security Tracker entry for `CVE-2026-27145`
19. Ubuntu Security entry for `CVE-2026-27145`